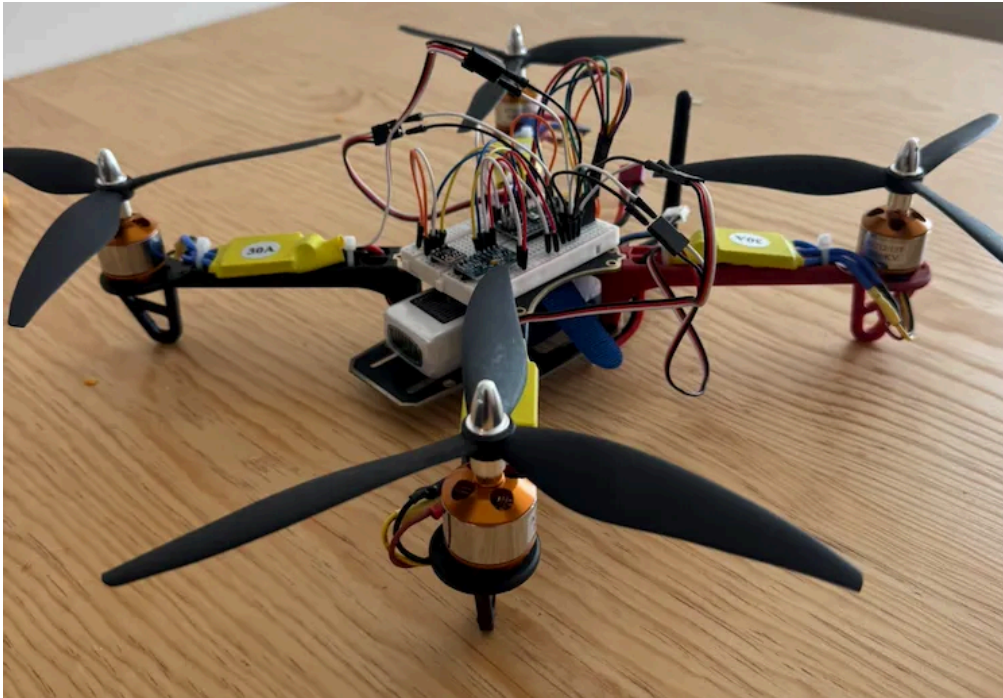
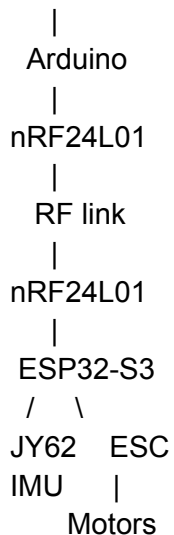


1.



Handheld Controller



I wanted to understand flight-control systems below the level of simply configuring an existing autopilot, so I started building a quadcopter control system from scratch. I built a handheld controller using an Arduino and nRF24L01 radio, wrote the receiver/flight-control software on an ESP32-S3, interfaced with an IMU over UART, and connected the controller to a 4-in-1 ESC for motor control. The most interesting part has been debugging the interfaces between systems—SPI radio communication, IMU packets, motor commands, and power electronics—because failures rarely stay neatly on the “hardware” or “software” side.

2. My setup has gotten increasingly command-line heavy as I've gotten deeper into embedded systems. I usually write C/C++ in VS Code and use Terminal for building, flashing, Git, and debugging. For PX4 development, I run the build environment in Docker on my Mac, use QGroundControl for configuration and initial hardware testing, and then use the PX4 NSH shell and serial output when I need to understand what's actually happening on the flight controller.

For smaller embedded projects, like my ESP32-based flight controller and Arduino-based transmitter, I tend to keep things simpler and use the Arduino IDE/serial monitor because the fast edit-flash-print-debug loop is useful. A lot of my debugging still comes down to strategically placed serial prints, checking raw sensor or radio data, and isolating one interface at a time.

My setup has evolved mostly out of necessity. I started with higher-level tools like the Arduino IDE and QGroundControl, but working on PX4 pushed me toward Linux, Docker, command-line builds, and reading source code. I've found that I like having high-level tools available for quick testing, but I don't want to depend on them when something breaks—I want to be able to go down a layer and figure out what's actually happening.

3. I'd first figure out what kind of reset is happening by checking the MCU's reset-cause register and adding logging to capture what happens immediately beforehand. Since it only occurs every few hours, I'd try to make it reproducible by running multiple boards and stressing individual subsystems.

My first hardware check would be the power rails with an oscilloscope to look for voltage dips or noise. If power looks clean and the reset points toward firmware, I'd investigate watchdog timeouts, blocking code, memory issues, and interrupts, isolating subsystems one at a time. My main goal would be to change one variable at a time until I can turn the intermittent failure into something reproducible.

4. One thing I've learned is that abstractions are incredibly useful until they prevent you from understanding what the hardware is actually doing. When I first started working with PX4 and embedded systems, it was tempting to treat interfaces as black boxes—call a driver, set a parameter, and expect hardware to respond. I've found that when something breaks, being willing to go underneath the abstraction and inspect packets, registers, electrical connections, timing, or the actual implementation is often much faster than repeatedly changing high-level code.

5. While building my flight controller, I had an IMU that would occasionally stop returning usable data even though the serial connection was still active. As a temporary fix, I added logic

to detect when valid packets stopped arriving and reinitialize the interface rather than trying to fully diagnose why it had gotten into that state.

I knew resetting the interface was treating the symptom rather than the root cause, but it let me keep testing the rest of the flight-control system. Once the rest of the system was working, I could come back and isolate the communication issue without blocking the entire project.